



COMPLETE BAZEL MASTERY GUIDE

From Absolute Beginner to Expert Level with C++
A Comprehensive 200+ Page Reference



Complete Table of Contents

PART I: FOUNDATIONS

Chapter 1: Introduction to Bazel (What, Why, When)

Chapter 2: Installation & Environment Setup

Chapter 3: Bazel Concepts & Terminology

Chapter 4: WORKSPACE Files Deep Dive

Chapter 5: BUILD Files Mastery

PART II: C++ WITH BAZEL

Chapter 6: C++ Build Rules (cc_library, cc_binary, cc_test)

Chapter 7: Header Files & Include Paths

Chapter 8: Linking & Dependencies

Chapter 9: Compiler Flags & Optimization

Chapter 10: Multi-Module C++ Projects

PART III: DEPENDENCY MANAGEMENT

Chapter 11: Internal Dependencies

Chapter 12: External Dependencies

Chapter 13: Third-Party Libraries

Chapter 14: Dependency Hell & Resolution

Chapter 15: Bzlmod (Modern Dependency Management)

PART IV: TESTING

Chapter 16: Unit Testing with Google Test

Chapter 17: Integration & System Tests

Chapter 18: Test Data & Resources

Chapter 19: Test Coverage & Analysis

PART V: ADVANCED TOPICS

Chapter 20: Platforms & Toolchains

Chapter 21: Cross-Compilation

Chapter 22: Custom Rules & Macros

Chapter 23: Aspects & Providers

Chapter 24: Remote Execution & Caching

Chapter 25: Build Performance Optimization

PART VI: REAL-WORLD SCENARIOS

Chapter 26: 10-Module Interconnected Project

Chapter 27: Common Mistakes & Solutions

Chapter 28: Migration from CMake/Make

Chapter 29: CI/CD Integration

Chapter 30: Best Practices & Patterns

PART I: FOUNDATIONS

Chapter 1: Introduction to Bazel

1.1 What is Bazel?

Bazel is a build and test tool developed by Google, open-sourced in 2015. It builds code in multiple languages and outputs binaries for multiple platforms. Bazel was born from Google's internal build tool called Blaze, which has been building Google's massive codebase for over a decade.

1.1.1 The Problem Bazel Solves

Traditional build systems like Make, CMake, or autotools face several challenges at scale:

- **Inconsistent Builds:** Different developers get different build results
- **Slow Incremental Builds:** Over-rebuilding when dependencies change
- **No Distributed Building:** Can't leverage multiple machines
- **Poor Dependency Management:** Implicit dependencies cause issues
- **No Reproducibility:** Can't recreate exact same binary

1.1.2 Core Philosophy

Bazel's Three Core Principles:

1. **Correctness:** Every build produces correct output
2. **Performance:** Builds are as fast as possible
3. **Reproducibility:** Same inputs always produce same outputs

1.1.3 How Bazel Achieves Speed

- **Incremental Builds:** Only rebuilds what changed
- **Parallel Execution:** Runs independent tasks in parallel
- **Caching:** Local and remote caching of build artifacts
- **Sandboxing:** Ensures builds are hermetic
- **Content-Based Hashing:** Tracks actual content, not timestamps

1.2 When Should You Use Bazel?

1.2.1 Ideal Use Cases

Scenario	Why Bazel Excels
Large codebases (1M+ lines)	Incremental builds save hours per day
Multiple languages	Unified build system for C++, Java, Python, Go, etc.
Monorepo architecture	Designed specifically for monorepos
Team > 10 developers	Consistent builds across team
Complex dependencies	Explicit dependency graph
Cross-platform builds	Built-in platform support

1.2.2 When NOT to Use Bazel

Consider alternatives if:

- Small project (<10k lines of code)
- Simple single-binary application
- Team unfamiliar with declarative build systems
- Existing complex CMake setup working well
- Need for rapid prototyping without build configuration

1.3 Bazel vs Other Build Systems

1.3.1 Bazel vs Make

```
# Makefile example
main: main.o utils.o
    g++ -o main main.o utils.o

main.o: main.cpp utils.h
    g++ -c main.cpp

utils.o: utils.cpp utils.h
    g++ -c utils.cpp
```

Issues with Make:

- Imperative style (describes HOW to build)
- Timestamp-based (not content-based)
- No built-in parallelism
- Global scope pollution

1.3.2 Bazel vs CMake

```
# CMakeLists.txt example
cmake_minimum_required(VERSION 3.10)
project(MyProject)

add_library(utils utils.cpp)
add_executable(main main.cpp)
target_link_libraries(main utils)
```

CMake Advantages: Widely adopted, extensive ecosystem, flexible

Bazel Advantages: Better caching, faster incremental builds, hermetic builds

1.3.3 Comparison Table

Feature	Bazel	CMake	Make
Incremental Builds	✓ Excellent	✓ Good	⚠ Basic
Parallel Execution	✓ Built-in	✓ Good	⚠ Manual (-j)
Remote Caching	✓ Native	✗ No	✗ No

Reproducibility	✔ Guaranteed	⚠ Depends	✗ No
Learning Curve	⚠ Steep	⚠ Moderate	✔ Easy
Ecosystem	⚠ Growing	✔ Mature	✔ Universal

1.4 Bazel Architecture

1.4.1 Build Process Overview

1. **Loading Phase:** Reads BUILD files, evaluates macros
2. **Analysis Phase:** Creates action graph from targets
3. **Execution Phase:** Runs actions to produce artifacts

Deep Dive: The Three Phases

Loading Phase:

- Parses BUILD and .bzl files
- Evaluates load() statements
- Creates package objects
- No execution of rules yet

Analysis Phase:

- Evaluates rule implementations
- Builds dependency graph
- Creates action graph
- Determines what needs building

Execution Phase:

- Executes actions in dependency order
- Uses sandboxing for hermetic builds
- Caches results
- Produces output artifacts

Chapter 2: Installation & Environment Setup

2.1 Installing Bazel

2.1.1 Understanding Bazelisk

Bazelisk is a wrapper for Bazel that automatically downloads and runs the correct version of Bazel. It's the recommended way to install Bazel.

Why Use Bazelisk?

- Automatic version management
- Team consistency (everyone uses same version)
- Easy version switching per project
- Handles Bazel updates smoothly

2.1.2 Linux Installation (Detailed)

```
# Method 1: Download binary directly
wget https://github.com/bazelbuild/bazelisk/releases/latest/download/bazel
chmod +x bazelisk-linux-amd64
sudo mv bazelisk-linux-amd64 /usr/local/bin/bazel

# Method 2: Using package managers
# Ubuntu/Debian with Bazel apt repository
sudo apt install apt-transport-https curl gnupg
curl -fsSL https://bazel.build/bazel-release.pub.gpg | gpg --dearmor >base
sudo mv bazel-archive-keyring.gpg /usr/share/keyrings
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/bazel-archive-keyring."
sudo apt update && sudo apt install bazel

# Method 3: Build from source
sudo apt install g++ unzip zip
git clone https://github.com/bazelbuild/bazel.git
cd bazel
bazel build //src:bazel

# Verify installation
bazel --version
which bazel
```

2.1.3 macOS Installation (Detailed)

```
# Method 1: Homebrew (recommended)
brew install bazelisk
brew install bazel # If you want specific version

# Method 2: Manual download
curl -Lo bazelisk https://github.com/bazelbuild/bazelisk/releases/latest/d
chmod +x bazelisk
sudo mv bazelisk /usr/local/bin/bazel

# For Apple Silicon (M1/M2)
curl -Lo bazelisk https://github.com/bazelbuild/bazelisk/releases/latest/d
chmod +x bazelisk
sudo mv bazelisk /usr/local/bin/bazel

# Verify
bazel --version
```

2.1.4 Windows Installation (Detailed)

```
# Method 1: Chocolatey
choco install bazelisk

# Method 2: Scoop
scoop install bazelisk

# Method 3: Manual
# Download from: https://github.com/bazelbuild/bazelisk/releases
# Add to PATH

# Method 4: MSYS2
pacman -S mingw-w64-x86_64-bazel

# Verify
bazel --version

# Windows-specific: Enable long paths
New-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem"
```

2.1.5 Version Management

Create a `.bazelversion` file in your project root:

```
# .bazelversion
6.4.0
```

Bazelisk will automatically use this version when building your project.

2.2 Prerequisites & Dependencies

2.2.1 Required Tools

Tool	Purpose	Installation
GCC/Clang	C++ compiler	<code>apt install build-essential</code>
Python 3	Bazel is written in Python	<code>apt install python3</code>
JDK 11+	Bazel runs on JVM	<code>apt install openjdk-11-jdk</code>

Git	Version control	<code>apt install git</code>
-----	-----------------	------------------------------

2.2.2 System Requirements

- **RAM:** Minimum 4GB, Recommended 8GB+
- **Disk:** 2GB+ free space for Bazel cache
- **CPU:** Multi-core recommended for parallel builds
- **OS:** Linux, macOS, Windows 10+

2.3 Initial Configuration

2.3.1 Creating .bazelrc

The .bazelrc file contains default Bazel settings. Create it in your home directory or project root:

```
# ~/.bazelrc (Global configuration)
# or
# .bazelrc (Project-specific)

# Enable color output
common --color=yes

# Show detailed build info
build --show_timestamps
build --show_progress_rate_limit=1

# Use all available cores
build --jobs=auto

# Default C++ standard
build --cxxopt=-std=c++17

# Enable compiler warnings
build --cxxopt=-Wall
build --cxxopt=-Wextra

# Optimization levels
build:debug --compilation_mode=dbg
build:release --compilation_mode=opt

# Remote cache (if available)
# build --remote_cache=grpc://cache.example.com:9092

# Disk cache location
build --disk_cache=~/.cache/bazel

# Test output
test --test_output=errors
test --test_summary=detailed

# Platform-specific configurations
build:linux --cxxopt=-fPIC
build:macos --macos_minimum_os=10.15
build:windows --enable_runfiles
```

2.3.2 Understanding .bazelrc Syntax

```
# Command syntax: command[:config] [--flag]

# Apply to all commands
common --flag

# Apply to specific command
build --flag
test --flag
run --flag
query --flag

# Named configuration
build:myconfig --flag1 --flag2

# Invoke with: bazel build --config=myconfig

# Import other rc files
import %workspace%/tools/bazel.rc

# Try-import (don't error if missing)
try-import %workspace%/.bazelrc.local
```

2.3.3 Environment Variables

```
# Set in shell profile (.bashrc, .zshrc, etc.)

# Bazel output directory
export OUTPUT_BASE="$HOME/.cache/bazel/output"

# Maximum heap size
export BAZEL_OPTS="-Xmx4g"

# Compiler path
export CC=/usr/bin/gcc-11
export CXX=/usr/bin/g++-11

# User-specific configuration
export BAZELRC="$HOME/.bazelrc"

# Disable analytics
export BAZEL_ANALYTICS=false
```

2.4 Editor/IDE Setup

2.4.1 Visual Studio Code

```
# Install extensions:
# 1. Bazel (bazelbuild.vscode-bazel)
# 2. C/C++ (ms-vscode.cpptools)
# 3. clangd (llvm-vs-code-extensions.vscode-clangd)

# .vscode/settings.json
{
  "bazel.executable": "bazel",
  "bazel.buildifierExecutable": "buildifier",
  "bazel.enableCodeLens": true,
  "C_Cpp.default.compileCommands": "${workspaceFolder}/compile_commands.
  "clangd.arguments": [
    "--compile-commands-dir=${workspaceFolder}",
    "--background-index"
  ]
}
```

2.4.2 CLion / IntelliJ

```
# Install Bazel plugin from JetBrains Marketplace
# File > Settings > Plugins > Search "Bazel"

# .bazelproject file in project root
directories:
  src
  tests

targets:
  //src/...
  //tests/...

derive_targets_from_directories: true

build_flags:
  --cxxopt=-std=c++17

test_flags:
  --test_output=errors
```

2.4.3 Generate compile_commands.json

For better IDE integration:

```
# Using compiledb (install: pip install compiledb)
bazel build //...
compiledb make

# Or using Bazel Compilation Database
# https://github.com/grailbio/bazel-compilation-database
git clone https://github.com/grailbio/bazel-compilation-database
cd bazel-compilation-database
python setup.py install

# Generate
bazel run @hedron_compile_commands//:refresh_all
```


Chapter 3: Bazel Concepts & Terminology

3.1 Core Concepts

3.1.1 Workspace

A workspace is a directory containing:

- A WORKSPACE or WORKSPACE.bazel file at the root
- Source code and BUILD files
- Build outputs (in bazel-* directories)

Key Points:

- One workspace per project
- All builds happen within workspace context
- Multiple projects = multiple workspaces (or monorepo)
- WORKSPACE file can be empty but must exist

3.1.2 Packages

A package is a directory containing a BUILD file. Each package:

- Contains a set of related targets
- Has a unique label path
- Can have sub-packages
- Is the unit of visibility control

```
workspace/
├─ WORKSPACE
├─ src/           # Package: //src
│  └─ BUILD
│  └─ lib/        # Package
```

